Solution: Longest Repeating Subsequence

Problem Statement

Given a sequence, find the length of its longest repeating subsequence (LRS). A repeating subsequence will be the one that appears at least twice in the original sequence and is not overlapping (i.e. none of the corresponding characters in the repeating subsequences have the same index).

Example 1:

Input: "t o m o r r o w"
Output: 2
Explanation: The longest repeating subsequence is "or" {tomorrow}.

Example 2:

Input: "a a b d b c e c"
Output: 3
Explanation: The longest repeating subsequence is "a b c" {a a b d b c e c}.

Example 3:

Input: "f m f f"
Output: 2
Explanation: The longest repeating subsequence is "f f" {f m f f, f m f f}. Please note the second last character is shared in LRS.

Basic Solution

The problem is quite similar to the `"Longest Common Subsequence" (LCS)`, with two differences:

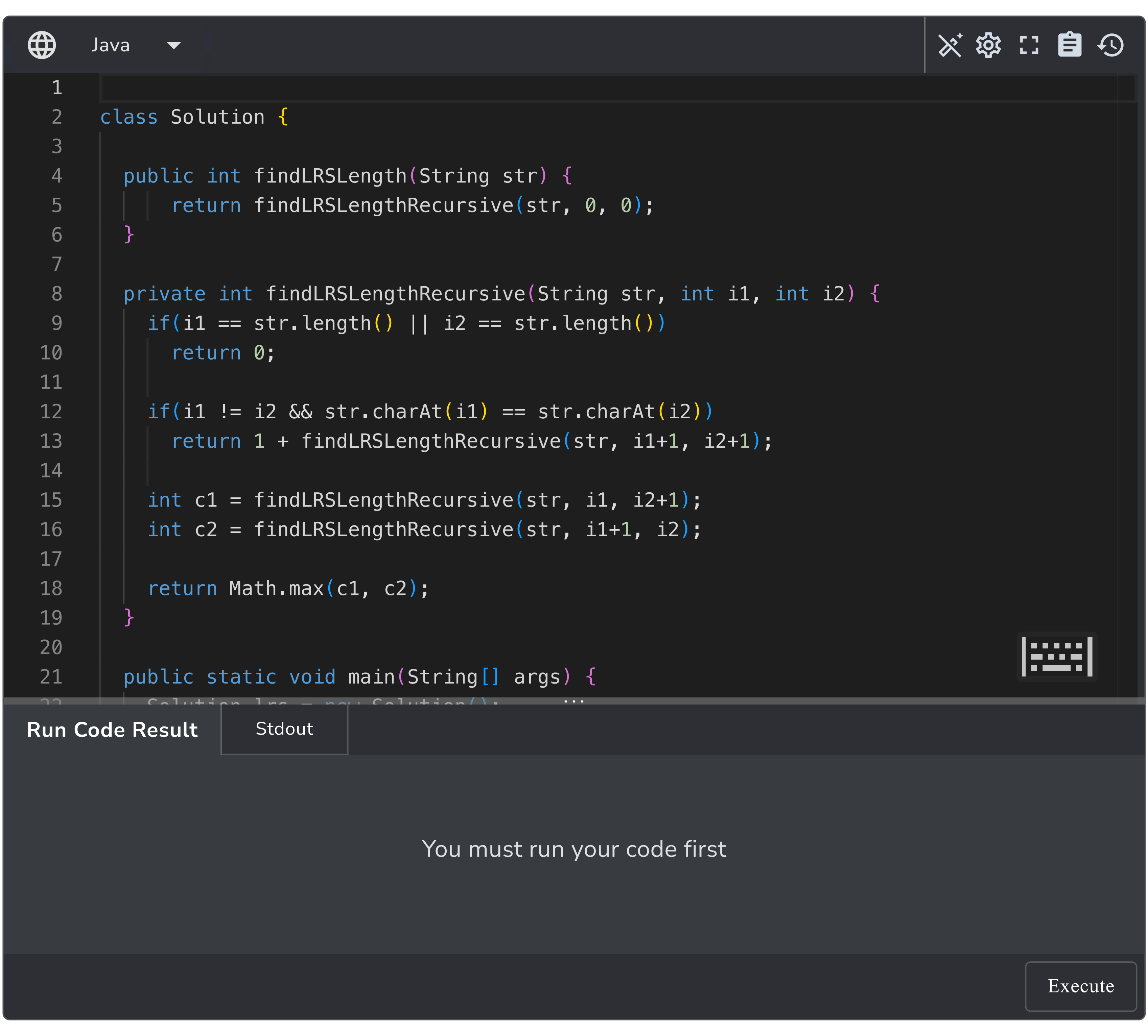
1. In LCS, we were trying to find the longest common subsequence between the two strings, whereas in LRS we are trying to find the two longest common subsequences within one string.
2. In LRS, every corresponding character in the subsequences should not have the same index.

A basic brute-force solution could be to try all subsequences of the given sequence to find the longest repeating one, but the problem is how to ensure that the LRS's characters do not have the same index. For this, we can start with two indexes in the given sequence, so at any step we have two choices:

1. If the two indexes are not the same and the characters at both the indexes are same, we can recursively match for the remaining length (i.e. by incrementing both the indexes).
2. If the characters at both the indexes don't match, we start two new recursive calls by incrementing each index separately. The LRS would be the one with the highest length from the two recursive calls.

Code

Here is the code:



```java

class Solution {

  public int findLRSLength(String str) {
    return findLRSLengthRecursive(str, 0, 0);
  }

  private int findLRSLengthRecursive(String str, int i1, int i2) {
    if(i1 == str.length() || i2 == str.length())
      return 0;

    if(i1 != i2 && str.charAt(i1) == str.charAt(i2))
      return 1 + findLRSLengthRecursive(str, i1+1, i2+1);

    int c1 = findLRSLengthRecursive(str, i1, i2+1);
    int c2 = findLRSLengthRecursive(str, i1+1, i2);

    return Math.max(c1, c2);
  }

  public static void main(String[] args) {
```

The time complexity of the above algorithm is exponential $O(2^n)$, where 'n' is the length of the input sequence. The space complexity is $O(n)$ which is used to store the recursion stack.

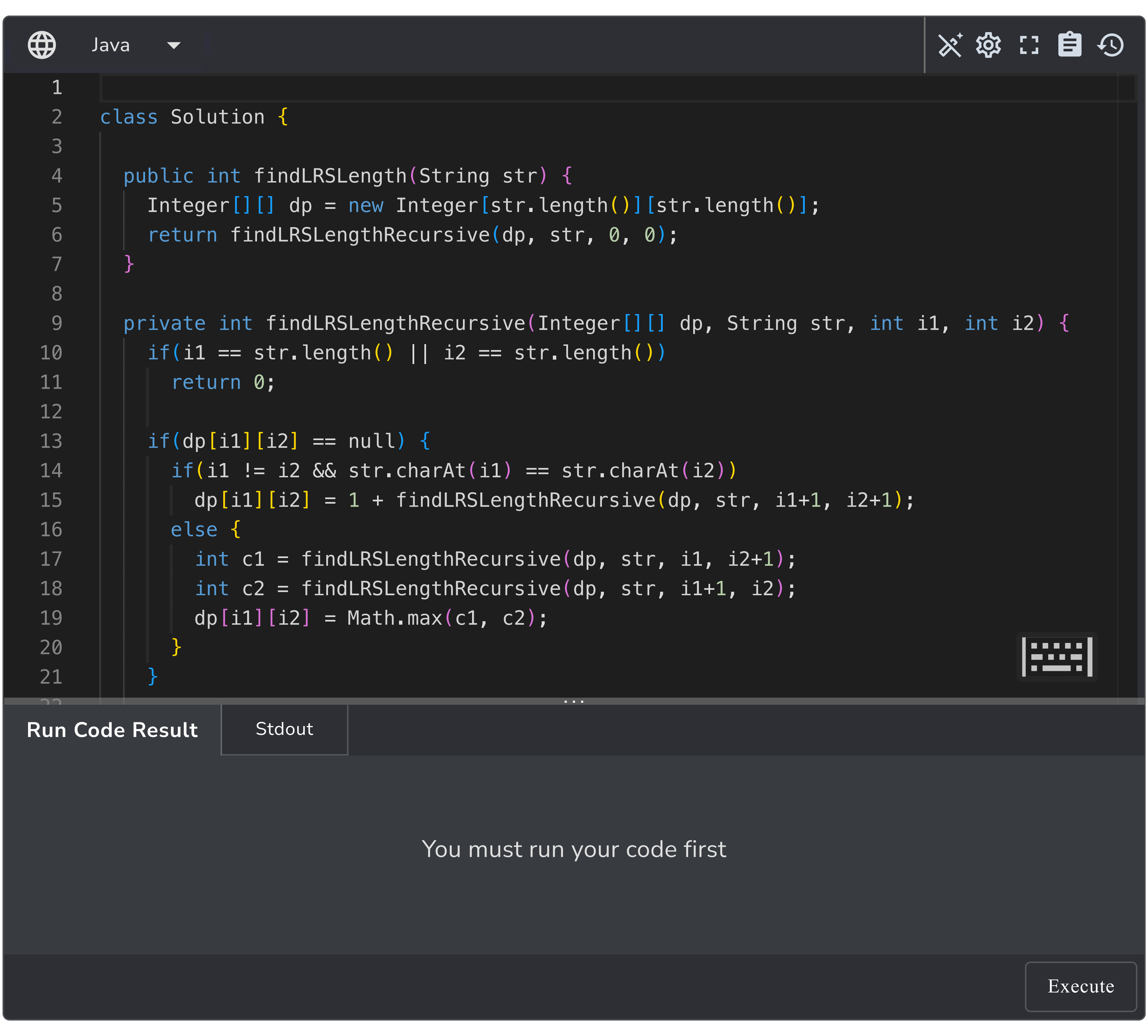
Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems.

The two changing values to our recursive function are the two indexes, i1 and i2. Therefore, we can store the results of all the subproblems in a two-dimensional array. (Another alternative could be to use a hash-table whose key would be a string (i1 + "|" + i2)).

Code

Here is the code:



```java

class Solution {

  public int findLRSLength(String str) {
    Integer[][] dp = new Integer[str.length()][str.length()];
    return findLRSLengthRecursive(dp, str, 0, 0);
  }

  private int findLRSLengthRecursive(Integer[][] dp, String str, int i1, int i2) {
    if(i1 == str.length() || i2 == str.length())
      return 0;

    if(dp[i1][i2] == null) {
      if(i1 != i2 && str.charAt(i1) == str.charAt(i2))
        dp[i1][i2] = 1 + findLRSLengthRecursive(dp, str, i1+1, i2+1);
      else {
        int c1 = findLRSLengthRecursive(dp, str, i1, i2+1);
        int c2 = findLRSLengthRecursive(dp, str, i1+1, i2);
        dp[i1][i2] = Math.max(c1, c2);
      }
    }
```

Bottom-up Dynamic Programming

Since we want to match all the subsequences of the given string, we can use a two-dimensional array to store our results. As mentioned above, we will be tracking two indexes to overcome the overlapping problem. So for each of the two indexes, 'i1' and 'i2', we will choose one of the following options:

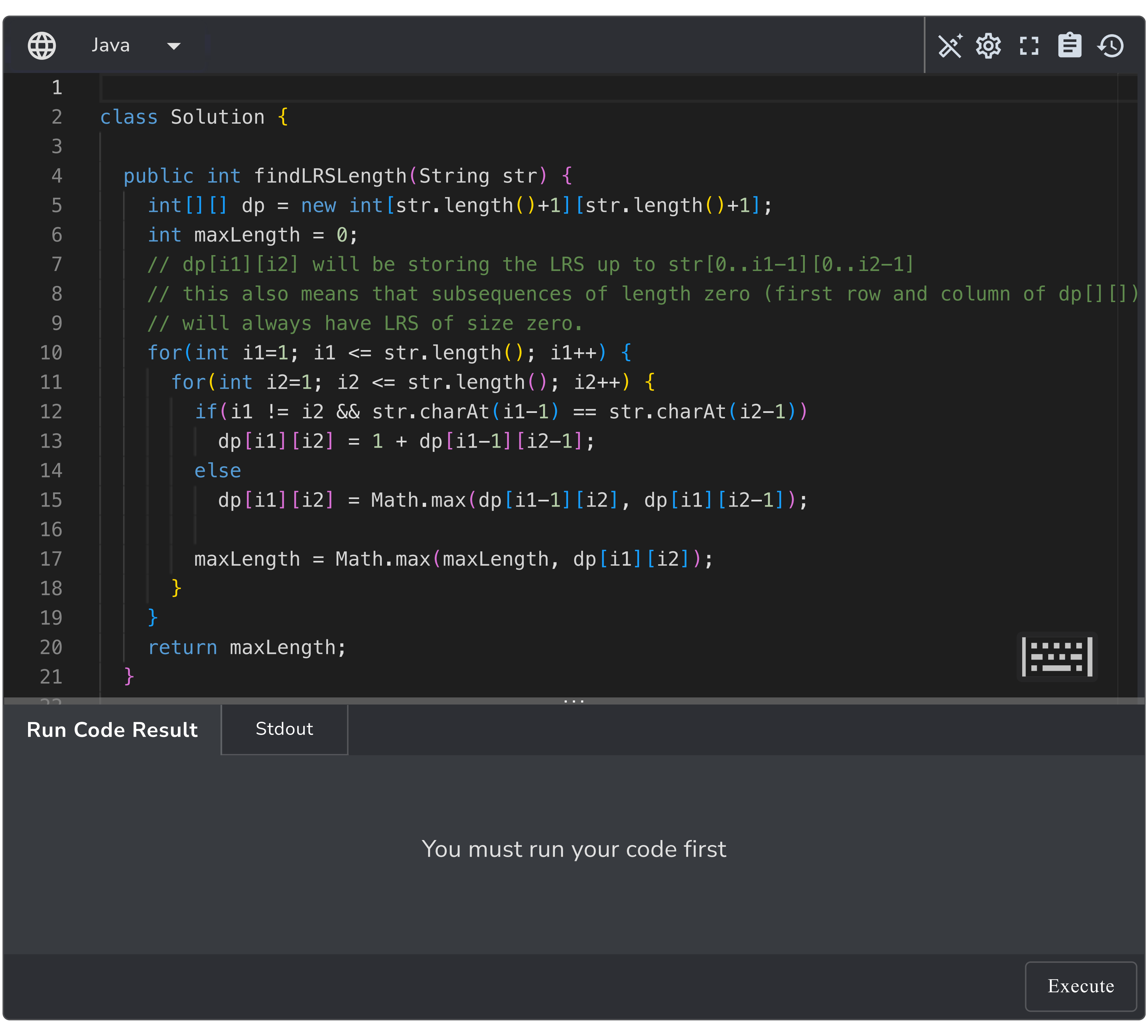
1. If 'i1' and 'i2' are different and the character `str[i1]` matches the character `str[i2]`, then the length of the LRS would be one plus the length of LRS up to `i1-1` and `i2-1` indexes.
2. If the character at `str[i1]` does not match `str[i2]`, we will take the LRS by either skipping 'i1'th or 'i2'th character.

So our recursive formula would be:

```
if i1 != i2 && str[i1] == str[i2]
  dp[i1][i2] = 1 + dp[i1-1][i2-1]
else
  dp[i1][i2] = max(dp[i1-1][i2], dp[i1][i2-1])
```

Code

Here is the code for our bottom-up dynamic programming approach:



```java

class Solution {

  public int findLRSLength(String str) {
    int[][] dp = new int[str.length()+1][str.length()+1];
    int maxLength = 0;
    // dp[i1][i2] will be storing the LRS up to str[0..i1-1][0..i2-1]
    // this also means that subsequences of length zero (first row and column of dp[][])
    // will always have LRS of size zero.
    for(int i1=1; i1 <= str.length(); i1++) {
      for(int i2=1; i2 <= str.length(); i2++) {
        if(i1 != i2 && str.charAt(i1-1) == str.charAt(i2-1))
          dp[i1][i2] = 1 + dp[i1-1][i2-1];
        else
          dp[i1][i2] = Math.max(dp[i1-1][i2], dp[i1][i2-1]);

        maxLength = Math.max(maxLength, dp[i1][i2]);
      }
    }
    return maxLength;
  }
```

The time and space complexity of the above algorithm is $O(n^2)$, where 'n' is the length of the input sequence.
